

run1-4.sh

You need to understand how the run1-4.sh script works to make your own script.

Learning Goals

- Explain the seven steps of `run1-4.sh` and why each is needed.
- Read the Bash constructs the script uses: `set -e`, `if !`, heredoc, and `sed`.
- Explain why the script is safe to rerun (create only if missing, no `migrate: fresh`).

What run1-4.sh Sets Up

1. Prepare: Create the Laravel project (only once)
2. Check: Is MySQL running?
3. Database: Create studentdb and the ase230 user if missing
4. Configure: Point Laravel's .env at studentdb
5. Copy: Copy the lesson files into the project
6. Migrate: Reuse or create the table, insert sample students
7. Verify: Print the API routes

Earlier, you generated the model and migration with

`make:model -m` and filled them in by hand.

- The script gets the same result by copying finished files (Step 5).
- This is the first script in the module that gives you real, persisted data instead of a fixed array in the controller.
- Unlike `run1-3.sh`, it connects to MySQL and shares the `studentdb` database with Module 1.

```
cd "4. Models and Database"  
bash ./run1-4.sh
```

Reading the Script: The Top

```
#!/bin/bash
set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SOURCE_DIR="$SCRIPT_DIR/student-api1-4"
TARGET_DIR="$SCRIPT_DIR/student-api"
DB_DATABASE="studentdb"
DB_USERNAME="ase230"
DB_PASSWORD="ase230pass"
```

- `set -e` : stop at the first failing command, so a broken step never leads to the next one.
- `SCRIPT_DIR` : the folder that contains the script, so it works from any directory. Read it inside-out: script path → folder → absolute path.
- `SOURCE_DIR` is the finished lesson code (input).
`TARGET_DIR` is the new Laravel project (output).
- Settings live in variables at the top: change them in one place. The three `DB_` values are the same ones `crud_pdo.php` uses.

Step 1: Create the Project Only Once

```
if [[ ! -f "$TARGET_DIR/artisan" ]]; then
    composer create-project laravel/laravel "$TARGET_DIR"
fi
cd "$TARGET_DIR"

if [[ ! -f routes/api.php ]]; then
    php artisan install:api
    --no-interaction
    --without-migration-prompt
fi
```

- `[[ ! -f X ]]` means "file X does not exist". If `artisan` exists, the project exists, so we skip.
Running the script twice is safe.
- `install:api` creates `routes/api.php` and installs Laravel Sanctum (this adds a `personal_access_tokens` migration).
- The two flags stop the command from asking questions.
- Always write `"$VAR"` with quotes: this folder name contains spaces.

What is `$TARGET_DIR/artisan`?

```
use Illuminate\Foundation\Application;
use Symfony\Component\Console\Input\ArgvInput;

define('LARAVEL_START', microtime(true));

// Register the Composer autoloader...
require __DIR__.'/vendor/autoload.php';

// Bootstrap Laravel and handle the command...
/** @var Application $app */
$app = require_once __DIR__.'/bootstrap/app.php';

$status = $app->handleCommand(new ArgvInput);

exit($status);
```

The `artisan` script loads Composer's autoloader, starts the Laravel application, passes it the command-line input, and exits with the command's status code.

Step 2: Is MySQL Running?

```
if ! mysqladmin -h 127.0.0.1 ping >/dev/null 2>&1; then
    echo "MySQL is not running on 127.0.0.1:3306" >&2
    exit 1
fi
```

- Every command returns an exit code: `0` means success, anything else means failure.
- `if ! command` runs the block when the command fails.
- `>/dev/null 2>&1` throws away all output. We only care about the exit code.
- `>&2` prints to the error stream. `exit 1` stops the script with a failure.
- `ping` succeeds whenever the server answers, even with `Access denied`. It checks that the server is alive, not that

Step 3: Create the Database and User

```
if [[ "$(uname -s)" == "Linux" ]]; then
    MYSQL_ROOT=(sudo mysql -u root)      # WSL2 / Linux
else
    MYSQL_ROOT=(env MYSQL_PWD="${MYSQL_ROOT_PASSWORD:-}" mysql -u root) # macOS
fi

if ! "${MYSQL_ROOT[@]}" <<SQL
CREATE DATABASE IF NOT EXISTS \`${DB_DATABASE}\`;
CREATE USER IF NOT EXISTS '${DB_USERNAME}'@'localhost' IDENTIFIED BY '${DB_PASSWORD}';
GRANT ALL PRIVILEGES ON \`${DB_DATABASE}\`.* TO '${DB_USERNAME}'@'localhost';
SQL
then
    echo "Cannot configure MySQL as root. ..." >&2
    exit 1
fi
```

- Here, `root` means the MySQL administrator, not the WSL2/macOS operating-system account.
- The script uses it once to create the database and user.
- `MYSQL_ROOT=(...)` stores the command in an array, so `"${MYSQL_ROOT[@]}"` runs `sudo mysql -u root` on

MySQL root

- The same SQL runs on both platforms. Only the way we log in as `root` differs.
- **WSL2 / Linux:** MySQL `root` uses `auth_socket`. It accepts only the Linux administrator (`sudo`), not a password. Running plain `mysql -u root` fails with `ERROR 1698 (28000): Access denied`. Module 1 also uses `sudo mysql`.
- **macOS (Homebrew):** `mysql -u root` usually works. Set `MYSQL_ROOT_PASSWORD` only if `root` has a password.
- Do **not** run the whole script with `sudo bash`: the Laravel project files would become owned by `root`. The script calls `sudo` for the one MySQL command only, so it may ask for your Linux password once.

Laravel Database access

- Laravel itself logs in as `ase230` , the Module 1 account, which can only touch `studentdb` .
- This is the **principle of least privilege**: the grant covers one database, not the whole server.
- `IF NOT EXISTS` makes the SQL safe to run again. If Module 1 already created `studentdb` and `ase230` , nothing is replaced and no data is lost.

Reading the Tricky Parts of Step 3

Syntax	Meaning
<code><<SQL ... SQL</code>	Heredoc: the lines in between are fed to <code>mysql</code> as input, as if you typed them.
<code>\$DB_DATABASE</code>	The shell fills in the value <i>before</i> MySQL sees the text.
<code>\`</code>	The backslash prevents shell command substitution, so MySQL receives literal backticks to quote the expanded database name.
<code>MYSQL_PWD="-" "mysql</code>	Sets the password for this

macOS, only if MySQL root has a password:

```
MYSQL_ROOT_PASSWORD=your_password bash ./run1-4.sh
```

WSL2 and macOS without a root password:

```
bash ./run1-4.sh
```

Step 4: Point .env at MySQL

Laravel's generated `.env` defaults to SQLite, but this project uses the MySQL database configured earlier. The script changes the connection setting:

```
DB_CONNECTION=sqlite
```

The `sed` command below changes it to

`DB_CONNECTION=mysql` and sets the MySQL connection details.

```
sed -E \  
-e 's/^DB_CONNECTION=.* /DB_CONNECTION=mysql/' \  
-e 's/^#? *DB_HOST=.* /DB_HOST=127.0.0.1/' \  
-e 's/^#? *DB_PORT=.* /DB_PORT=3306/' \  
-e "s/^#? *DB_DATABASE=.* /DB_DATABASE=$DB_DATABASE/" \  
-e "s/^#? *DB_USERNAME=.* /DB_USERNAME=$DB_USERNAME/" \  
-e "s/^#? *DB_PASSWORD=.* /DB_PASSWORD=$DB_PASSWORD/" \  
-e 's/^SESSION_DRIVER=.* /SESSION_DRIVER=file/' \  
-e 's/^CACHE_STORE=.* /CACHE_STORE=file/' \  
-e 's/^QUEUE_CONNECTION=.* /QUEUE_CONNECTION=sync/' \  
.  
mv .env.tmp .env
```

Pattern	Meaning
<code>-E</code>	Enables extended regular expressions, where <code>?</code> means "optional."
<code>-e</code> <code>'s/pattern/replacement/'</code>	Adds one substitution rule; repeat <code>-e</code> to change multiple settings.
<code>s/old/new/</code>	Replace <code>old</code> with <code>new</code>
<code>^</code>	Start of the line
<code>#? *</code>	An optional <code>#</code> , then optional spaces
<code>.*</code>	The rest of the line

Use sed for macOS and Linux (WSL2)

This works on macOS and WSL2 because both BSD `sed` and GNU `sed` support `-E` and repeated `-e` options.

- The script writes the result to `.env.tmp` and renames it, instead of using `sed -i`, whose syntax differs between platforms.

Before and After

- A new Laravel `.env` may have `# DB_HOST=127.0.0.1` commented out; the pattern matches it and writes an active `DB_HOST=127.0.0.1` setting.
- The full script also sets `DB_CONNECTION` , `DB_PORT` , `DB_DATABASE` , `DB_USERNAME` , and `DB_PASSWORD` for MySQL, plus `SESSION_DRIVER=file` , `CACHE_STORE=file` , and `QUEUE_CONNECTION=sync` so Laravel does not add service tables to the shared database.

This is the original:

```
DB_CONNECTION=sqlite
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=laravel
# DB_USERNAME=root
# DB_PASSWORD=
SESSION_DRIVER=database
QUEUE_CONNECTION=database
CACHE_STORE=database
```

This is the updated one:

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=studentdb  
DB_USERNAME=ase230  
DB_PASSWORD=ase230pass  
SESSION_DRIVER=file  
QUEUE_CONNECTION=sync  
CACHE_STORE=file
```

Why Not Run the artisan commands in the Script?

The script is a shortcut that builds the finished example.

- In your own project, you type the `artisan` commands yourself. Both routes end with the same files:

	By hand (the lesson)	<code>run1-4.sh</code>
Model and migration	Generate both, then edit the files	Copy the finished files
Controller	Generate it, then add the action methods	Copy the finished controller
Create the table	Apply the migration	The script runs the migration

Commands used in the lesson:

```
php artisan make:model Student -m
php artisan make:controller Api/StudentController --api
php artisan migrate
--path=database/migrations/<timestamp>_create_students_table.php
```

Why the script copies instead of generating:

1. `-m` creates a migration whose filename includes the **current time**.
 - Running `make:model Student -m` more than once can create multiple migrations for `students`, causing a `Table already exists` error when both are applied.
2. `make:model` and `make:controller` will not overwrite an existing file, so a rerun could not update it.

The generated files are empty and must be edited anyway.

- Copying the finished file does both jobs in one step.
- For your project, you need to add code to the automatically generated skeleton code.

Step 5: Copy the Lesson Files

```
mkdir -p app/Http/Controllers/Api  
cp "$SOURCE_DIR/app/Models/Student.php" app/Models/Student.php
```

The pattern is `cp "$SOURCE_DIR/path" path` : copy a finished file from the lesson folder into the same place in the new project.

- `mkdir -p` creates the folder if it is missing and does not fail if it exists.
- `make:controller Api/StudentController` would have created `Api/` for you, but `cp` does not, so the script needs `mkdir -p`.

File	Role
Api/StudentController.php	index , show , store actions
Models/Student.php	Model with \$timestamp and \$fillable
routes/api.php	apiResource route
2024_01_01_000000_create_students_table.php	Table definition

Steps 6 and 7: Migrate, Seed, Verify

```
php artisan migrate  
  --path=database/migrations/2024_01_01_000000_create_students_table.php  
  --force
```

- `migrate --path=...` runs only the students migration. Finished migrations are skipped, so the script keeps your data when you rerun it.
- **Avoid `migrate:fresh` here.**
 - It drops every table in `studentdb`, including the `students` rows that Module 1 created.
- `--force` skips the confirmation prompt.

```
php artisan db:seed --class=StudentSeeder --force  
php artisan route:list --path=api
```

- `db:seed --class=StudentSeeder` adds the sample students.
- Inside `StudentSeeder`, `Student::firstOrCreate()` skips emails that already exist, so rerunning `db:seed` does not add duplicate students.
- `route:list --path=api` shows the API URLs so you can confirm the routes were registered.

```

class StudentSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        // Sample students data
        $students = [
            [
                'name' => 'John Doe',
                'email' => 'john.doe@university.edu',
                'age' => 20,
                'major' => 'Computer Science',
                'year' => 2
            ],
            ...
        ];
        foreach ($students as $studentData) {
            Student::firstOrCreate(['email' =>
                $studentData['email']], $studentData);
        }
    }
}

```

Read the APIs

- Read `student-api1-4/app/Http/Controllers/Api/StudentController.php` to find all the available APIs for this project.
- Read `student-api1-4/routes/api.php` to find how the controller methods are connected with the routes.
- The complete code has more APIs than these slides (`search`, `stats`, and others). Its responses also wrap the data in `success`, `message`, and `data`; the slide code returns the data directly.

Run the Server & Test

```
php artisan route:list --path=api
php artisan serve --port=8080
# WSL2: php artisan serve --host=0.0.0.0 --port=8080
```

Part of the `route:list` output:

GET HEAD	api/students	Api\StudentController@index
POST	api/students	Api\StudentController@store
GET HEAD	api/students/{student}	Api\StudentController@show
PUT	api/students/{student}	Api\StudentController@update
DELETE	api/students/{student}	Api\StudentController@destroy

Read Records

The seeder added 8 students to `studentdb` .

```
curl http://localhost:8080/api/students  
curl http://localhost:8080/api/students/1
```

```
{"success":true,"message":"Students retrieved successfully",  
"data":[{"id":1,"name":"John Doe", ...}, ...],"total":8}
```

- `total` counts every row in `students` , including rows
Module 1 added.
- `/api/students/1` returns the student with ID 1 (John Doe
on a fresh table).

```
curl -i -H 'Accept: application/json' http://localhost:8080/api/students/999
```

```
HTTP/1.1 404 Not Found
Host: localhost:8080
Connection: close
X-Powered-By: PHP/8.4.26
Cache-Control: no-cache, private
Date: Wed, 30 Sep 2026 15:20:17 GMT
Content-Type: application/json
Access-Control-Allow-Origin: *
```

```
{
  "message": "No query results for model [App\\Models\\Student] 999",
  "exception": "Symfony\\Component\\HttpKernel\\Exception\\NotFoundHttpException",
  "file": "student-api/vendor/laravel/framework/src/Illuminate/Foundation/Exceptions/Handler.php",
```

- `/api/students/999` returns `404` : route model binding finds no record.

Save a Record

```
curl -i -X POST http://localhost:8080/api/students \  
-H 'Accept: application/json' \  
-H 'Content-Type: application/json' \  
-d '{"name":"Grace Lee","email":"grace.lee@university.edu","age":20,"major":"Computer Science","year":2}'
```

```
HTTP/1.1 201 Created  
Host: localhost:8080  
Connection: close  
X-Powered-By: PHP/8.4.26  
Cache-Control: no-cache, private  
Date: Wed, 30 Sep 2026 15:22:35 GMT  
Content-Type: application/json  
Access-Control-Allow-Origin: *
```

```
{"success":true,"message":"Student created successfully",  
  "data":{"name":"Grace Lee","email":"grace.lee@university.edu","age":20,  
    "major":"Computer Science","year":2,"id":9}}%
```

- The response is `201 Created` and includes the new `id`.
The examples below call it `9`: use the ID you received.
- Send the same POST again: `422`, because `email` must be unique.
- Read it back: `curl`
`http://localhost:8080/api/students/9`

Verify Persistence

1. Stop the server with `Ctrl+C` and start it again.
2. `curl http://localhost:8080/api/students/9` : Grace Lee is still there.

Read MySQL DB directly

Look at the table directly (the password is ase230pass):

```
mysql -u ase230 -p studentdb -e "SELECT id, name, age, major, year FROM students"
```

id	name	age	major	year
1	John Doe	20	Computer Science	2
2	Jane Smith	21	Mathematics	3
3	Bob Johnson	19	Physics	1
4	Alice Brown	22	Chemistry	4
5	Charlie Wilson	20	Computer Science	2
6	Diana Davis	21	Biology	3
7	Eva Martinez	19	Engineering	1
8	Frank Garcia	23	Mathematics	4
9	Grace Lee	20	Computer Science	2

- The data lives in MySQL, not in the PHP code, so it survives a restart.
- Running `run1-4.sh` again keeps it: the script uses `migrate`, not `migrate:fresh`.

Update

With the complete lesson code:

```
curl -X PUT http://localhost:8080/api/students/9 \  
  -H 'Accept: application/json' -H 'Content-Type: application/json' \  
  -d '{"major":"Mathematics"}'
```

- PUT sends only the field to change: the sometimes validation rule allows it.

```
{"success":true,"message":"Student updated successfully",  
  "data":{"id":9,"name":"Grace Lee",  
    "email":"grace.lee@university.edu",  
    "age":20,"major":"Mathematics","year":2}}%
```

Delete

```
curl -X DELETE http://localhost:8080/api/students/9 -H 'Accept: application/json'
```

```
{"success":true,"message":"Student deleted successfully"}%
```

```
curl -i -H 'Accept: application/json' http://localhost:8080/api/students/9
```

- After `DELETE` , the last GET returns `404` .

Command Summary

Students should remember the `artisan` commands that generate the code and build the table:

```
php artisan make:model Student -m  
php artisan make:controller Api/StudentController --api  
php artisan migrate --path=database/migrations/<timestamp>_create_students_table.php
```

- In the script, the files are copied, instead of using the artisan command.

```
php artisan make:seeder StudentSeeder
php artisan db:seed --class=StudentSeeder
php artisan migrate:status # which migrations have run
php artisan route:list --path=api
# macOS
php artisan serve --port=8080
# Linux / WSL2
php artisan serve --host=0.0.0.0 --port=8080
```

- Do not run `php artisan migrate:fresh` on `studentdb`:
it drops every table, including Module 1's `students` data.

Bash Cheat Sheet from run1-4.sh

Syntax	Meaning
<code>set -e</code>	Stop at the first error
<code>\$(command)</code>	Use the command's output as a value
<code>"\$VAR"</code>	The variable's value (quotes protect spaces)
<code>\${VAR:-x}</code>	<code>VAR</code> , or <code>x</code> if it is unset or empty
<code>[[-f F]]</code>	<code>F</code> is an existing file
<code>! command</code>	Negate: true when the command fails
<code>a && b</code>	Run <code>b</code> only if <code>a</code> succeeds
<code>> file</code>	Write output to a file

When the Script Fails

Message	Likely cause	What to do
<code>MySQL is not running on 127.0.0.1:3306</code>	The server is stopped	Start MySQL, then rerun
<code>ERROR 1698 ... Access denied for user 'root'</code>	Old script on WSL2: root needs <code>sudo</code> (socket auth)	Use the updated script, or run <code>sudo mysql</code> by hand
	WSL2:	